

实验三 计数器和时钟

2023 年秋季学期

On receiving an interrupt, decrement the counter to zero.

– “Count Zero” 《零伯爵》，威廉·吉布森

在数字系统中，常用计数器来记录系统的工作状态，本实验复习了计数器的工作原理，通过介绍几种简单计数器的工作过程和设计方法、以及开发板系统时钟的使用，学习计数器的设计和定时器的工作原理。

3.1 加法计数器

利用触发器可以构成简单的计数器。图 3-1 是由 3 个上升沿触发的 D 触发器组成的 3 位二进制异步加法计数器，即在每个 Clock 的上升沿，计数器输出 $Q_2Q_1Q_0$ 加 1。

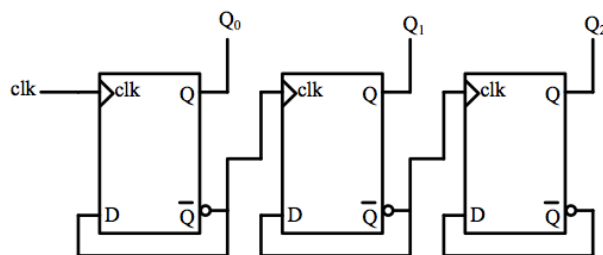


图 3-1: 3 位二进制加法计数器

图 3-2 是此 3 位二进制异步加法计数器的状态转移图。

也可以给 D 触发器加上“清零”和“置数”端，构成一个可以清零和置数的二进制异步加法计数器。请读者自行设计和验证此电路。

3.2 减法计数器

利用 D 触发器同样可以构成减法计数器，图 3-3 是由 3 个上升沿触发的 D 触发器组成的 3 位二进制异步减法计数器

图 3-4 是此 3 位二进制异步减法计数器的状态转移图。

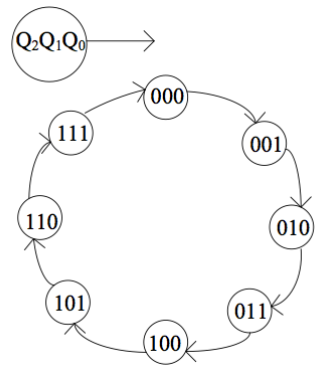


图 3-2: 3 位二进制加法计数器状态图

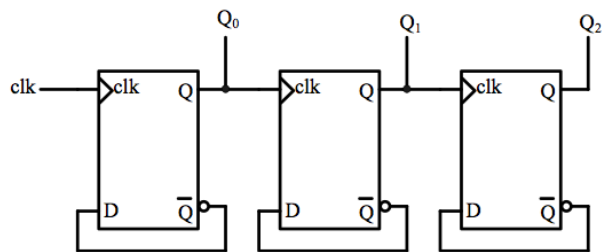


图 3-3: 3 位二进制异步减法计数器

利用 Verilog 语言可以方便的构建计数器，表 3-1 就是一个 3 位二进制减法计数器，也可以用类似的代码构成加法计数器。

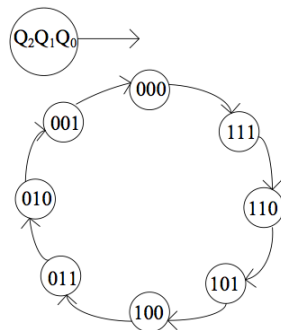


图 3-4: 3 位二进制减法计数器状态图

表 3-1: 3 位二进制带使能端的减法计数器代码

```
1 module vminus3(clk,en,out_q);
2     input  clk;
3     input  en;
4     output reg [2:0] out_q;
5
6     always @ (posedge clk)
7         if (en)    out_q <= out_q -1;
8         else      out_q <= 0;
9 endmodule
```

表 3-1 构建的减法计数器的仿真图如图 3-5 所示。

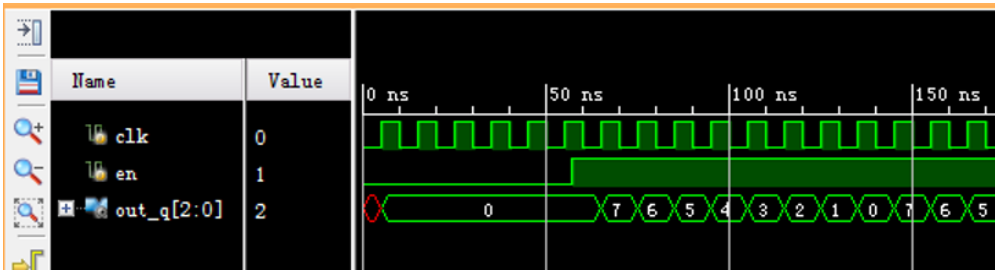


图 3-5: 减法计数器仿真图

带有时钟的模块仿真：带有时钟的时序电路仿真时，需要时钟信号不停的翻转，因此仿真代码在 initial 部分应使用 forever 语句对时钟输入进行改变，如下所示：

```
1     initial
2     begin
3         clk=1'b0;
4         forever
5             #5 clk=~clk;
6     end
```

这样可以产生一个周期为 10 个单位的时钟信号。

在仿真计数器的时候时常会出现输出值一直是未定义，即 XXXX 的情况。这主要是由于计数器的值或输出在仿真开始时是未定义的。在累加或翻转输出时对未定义的值进行操作结果还是 XXXX。这时需要在计数器代码内部增加 initial 语句，对计数值和输出结果进行初始化。

3.3 定时器

如果在计数器的时钟输入端输入一个固定周期的时钟，那么计数器就变成了定时器。

本实验的目的是学习 FPGA 开发平台上时钟源的使用，并结合计数器的设计方法学习定时器的设计。

3.3.1 开发板上的时钟信号

Nexys 开发板有一个能产生频率为 100MHz（周期为 10ns）时钟信号的振荡器，这个时钟信号连接到 FPGA 的 E3 引脚上，可以为用户的逻辑电路提供时钟。

为了使用该时钟，我们可以将缺省约束 XDC 文件中的时钟部分取消注释：

```
1 set_property -dict {PACKAGE_PIN E3 IOSTANDARD LVCMOS33} [get_ports {CLK100MHZ}];
```

这样在顶层文件中添加输入信号 CLK100MHZ 即是对应了板载 100MHz 的时钟。将此时钟信号作为计数器的时钟信号，即可构成一个定时器：

$$\text{计时时间} = \text{脉冲个数} \times \text{脉冲周期}$$

3.3.2 利用计数器进行时钟信号分频

利用开发板上提供的频率为 100MHz 时钟信号和定时器，我们可以设计任何我们需要的时钟信号。如下实例是产生周期为 1 秒的时钟信号的参考代码。其中 clk 是系统时钟，clk_1s 是产生的周期为 1 秒的时钟。

表 3-2: 1 秒时钟生成代码

```
1 always @(posedge clk)
2     if(count_clk==49999999)
3     begin
4         count_clk <=0;
5         clk_1s <= ~clk_1s;
6     end
7     else
8         count_clk <= count_clk+1;
```

请阅读、理解此段代码，并请思考为了能满足 0~49999999 的计数要求，变量“count_clk”的宽度如何设定？

为了更加方便地实现分频，我们还可以采用参数化的分频模块来实现分频器，代码如下：

表 3-3: 参数化分频器

```

1 module clkgen(input clkkin, input rst, input clken, output reg clkout);
2     parameter clk_freq=1000;
3     parameter countlimit=100000000/2/clk_freq-1;
4     reg[31:0] clkcount;
5     always @ (posedge clkkin)
6     if(rst)
7     begin
8         clkcount<=0;
9         clkout<=1'b0;
10    end
11    else
12    begin
13        if(clken)
14        begin
15            if(clkcount>=countlimit)
16            begin
17                clkcount<=32'd0;
18                clkout<=~clkout;
19            end
20            else
21            begin
22                clkcount<=clkcount+1;
23            end
24        end
25    end
26 end
27 endmodule

```

该参数化分频器使用了 Verilog 的 parameter 来对输出频率进行定义，并且在编译时自动计算常量的 countlimit，这样在实例化时可以利用

```
1 clkgen #(1) my1s_clk(CLK100MHZ,SW[0],1'b1,clk_1s);
```

直接指定输出的时钟频率。

3.4 数码管的动态显示

在实验 2 中我们曾使用过开发板上的七段数码管，根据数码管的硬件连接我们发现，每个数码管都有八个 LED 组成，分别标识为 CA、CB、CC、CD、CE、CF 和 CG，小数点标识为 DP。这些 LED 又和 FPGA 的固定引脚相连，如 CA 和 L3 相连，DP 和 M4 相连，而且八个数码管的所有 CA 端同时和一个

FPGA 的引脚 L3 相连的。如果我们将 FPGA 的 L3 引脚输出一个电平，则此电平是同时送到所有数码管的 CA 引脚的，如图 3-6 所示。当然，我们就可以通过 M1、L1、N4、N2、N5、M3、M6 和 N6 端输出低电平选中某个数码管来显示某个数据，但是当我们选择多个数码管时，这些数码管会同时显示相同的数据，因为他们的 CA、CB、CC、CD、CE、CF、CG 和 DP 都是连接在一起的。

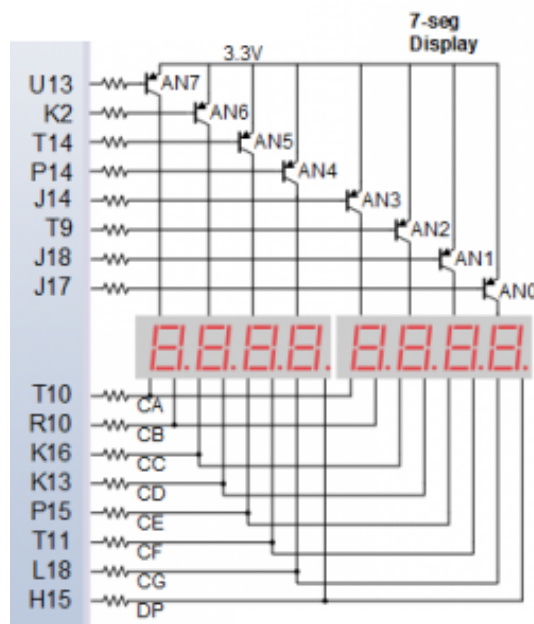


图 3-6: Nexys A7-100T 数码管连接

如果我们想让每个数码管显示不同的数字怎么办呢？比如我们想利用最右面的两个数码管显示“10”这两个字符。假设这两个数码管我们分别标识为 AN1 和 AN0，我们要在 AN1 上显示“1”这个字符，在 AN0 上显示“0”这个字符。这时，我们就要对数码管进行动态显示了。

具体方法如下：在某个时刻，我们只选中 AN1 让其显示“1”这个字符，在下一个时刻我们又只选中 AN0 让其显示“0”这个字符。然后再选 AN1 显示“1”，再选 AN0 显示“0”，这样我们就能看见“1”和“0”这两个字符在两个数码管上轮流显示，如果轮流的时间足够短，也就是两个数码管切换的足够快，根据人眼的视觉残留原理，看起来，“1”和“0”这两个字符就是同时在两个数码管上被显示的了。如果只要显示两位数字，可以用如下参考代码实现。

```
1 always @(select)
2     begin
```

```
3     case(select)
4         1'b0:begin seg7[7:0]=8'b11111110; display =out_dis0; end
5         1'b1:begin seg7[7:0]=8'b11111101; display =out_dis1; end
6     endcase
7 end
```

注意，这里的 `seg7` 及 `display` 都应该是 `reg` 型信号，可以分别对应输出的 AN 及 HEX。上述代码根据 `select` 信号分别选择数码管的选通信号及显示的内容，所以可以在两个数码管上轮流显示不同数字。这里，`select` 信号可以用一个高速的时钟来生成，例如，上述的 100MHz 的时钟。如果时钟频率偏低，会出现数码管闪烁的现象。当需要同时显示 8 位数字时，可以采用 3 位的 `select` 信号，利用计数器来轮流选通，请同学们自行根据前面所学的译码器及选择器来组合设计多数码管显示电路。

3.5 实验验收内容

3.5.1 上板验收（基础）

请在 Nexys A7-100T 开发板上实现一个计时器，在七段数码管上直接以十进制显示。

利用开发板上的频率为 100MHz 的时钟，请先设计一个分频器，其输入为 100MHz 的时钟，输出为一个频率为 1Hz，周期为 1 秒的时钟信号。再用这个新的频率为 1Hz 的时钟信号作为你设计的时钟信号，进行计数。

要求此计时器有开始、暂停和清零功能，要求从 00 计数到 59，计数值到 60 后重新从零开始计数。在数码管上用两位数字显示。

可以在计时结束的时候让某一个发光二极管闪烁一个时钟周期，提示计时结束。

3.5.2 上板验收（拓展）

在 Nexys A7-100 开发板上实现一个电子时钟，时钟要求能够显示时、分、秒；还可以有以下功能：调整时间；闹铃（在特定时间 LED 闪烁）；秒表（提供百分之一秒精度，可以停止重启）等。

3.5.3 在线测试

必做 计数器级联

选做 汉明码纠错